Distributed Tracing Across Services

follow one request through every hop

Watch the 4-min video

In one line: a trace is one request's whole journey and a span is one timed unit of work in it; services pass the trace context (the W3C `traceparent` header) on every call and message, so all spans join into one tree that shows exactly where the time goes.

1 Four seconds, eight services, everyone says "not us"

Checkout takes four seconds and touches eight services. Every team says its service is fast. Logs show fragments, but nobody can see the whole journey. **Distributed tracing** connects every hop of one request into a single picture.

2 Traces and spans

A **trace** is the full journey of one request, identified by a 16-byte trace ID. A **span** is one unit of work inside it (an HTTP call, a DB query, a message handler) with a name, start time, duration, attributes, status and a **parent span**. Together the spans form a tree, drawn as a **waterfall** timeline: what runs in sequence, what runs in parallel, and which span is slow.

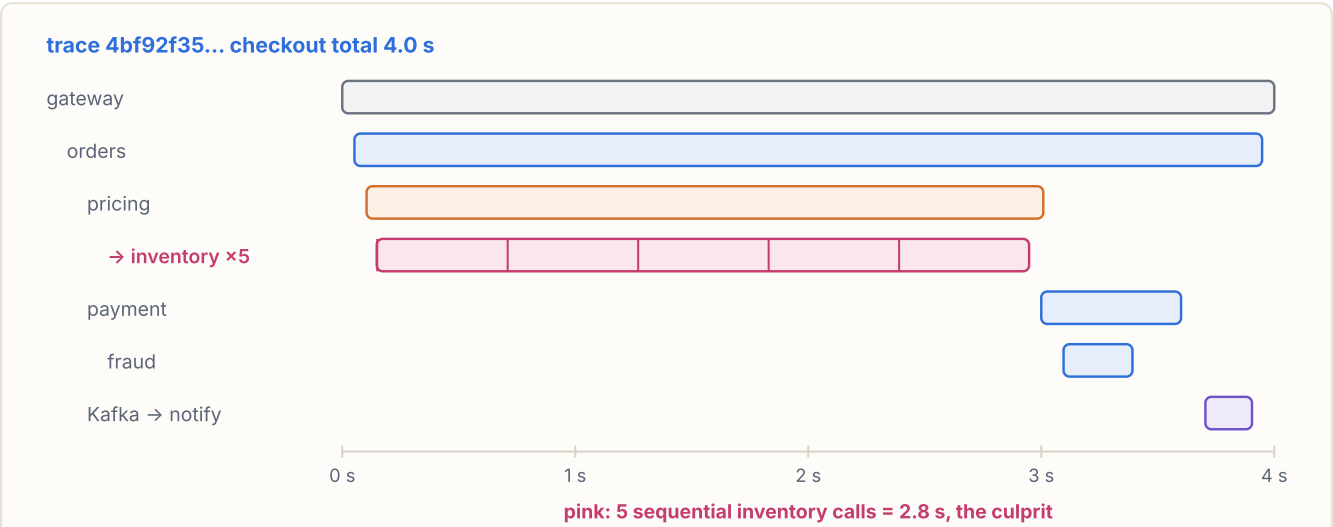


Figure 1. The waterfall from the worked example: pricing calls inventory five times in a row.

3 Context propagation

When service A calls B, it must pass the trace ID and its own span ID. The standard is the **W3C Trace Context** header `traceparent`: version, trace ID, parent span ID and flags (01 = sampled). B reads it, starts a child span, and passes its own context on. A companion `tracestate` header carries vendor-specific data.

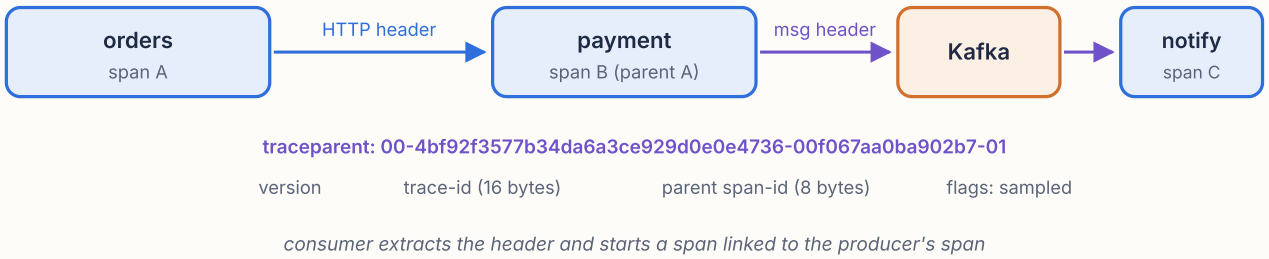


Figure 2. The same context travels in HTTP/gRPC headers and in message headers, so the trace survives the asynchronous hop.

Across Kafka there is no live call. The producer writes `traceparent` into the **message headers**; the consumer extracts it and starts a span linked to the producer's span, even if it runs minutes later. OpenTelemetry's messaging conventions model this with parent/child or span links.

4 In practice: OpenTelemetry

- **Instrument** with OpenTelemetry SDKs and auto-instrumentation for frameworks, HTTP clients, DB drivers and Kafka clients; they create spans and propagate context for you (lesson 64).
- **Export** via OTLP, often through an OpenTelemetry Collector, to a backend such as Jaeger, Grafana Tempo, Zipkin or Azure Monitor.
- **Sample** to control cost: head sampling keeps a fixed share at the start; **tail sampling** in the Collector can keep every error and slow trace plus a small share of normal ones.
- **Add attributes** such as order ID and tenant, so traces are searchable, and put the **trace ID in every log line** to jump from a log to its trace (lesson 65).

WHERE TRACES BREAK

- A thread pool, async callback or background job that loses the current context.
- A custom HTTP client or proxy that doesn't forward `traceparent`.
- An uninstrumented service, leaving a gap and orphaned child spans.
- Inconsistent sampling decisions, so half a trace is kept and half dropped.

WORKED EXAMPLE: FINDING 2.8 SECONDS

An order passes the gateway, orders, pricing, inventory, payment, fraud, Kafka and notifications. The waterfall shows **pricing calling inventory five times in sequence**, about 0.56 s each, 2.8 s in total. The team replaces them with one batch call for all items, and checkout drops from 4.0 s to **1.2 s**. No log search would have shown the five calls as one pattern.

5 Key terms

Trace: all spans of one request, sharing a trace ID

traceparent: W3C header: version-traceid-parentid-flags

Tail sampling: decide what to keep after the trace completes

Span: one timed unit of work with a parent

Context propagation: passing trace context on every call or message

Span link: connects spans across async or batch boundaries

6 Interview questions

Q1. What is a span? BASIC

One timed unit of work (HTTP call, DB query, message handler) with a start, duration, attributes and a parent; spans together form the trace tree.

Q2. What does the traceparent header contain? BASIC

A version, the 16-byte trace ID, the 8-byte parent span ID and trace flags such as sampled, e.g. 00-<trace-id>-<span-id>-01.

Q3. How do you trace a request that crosses a Kafka topic? SCENARIO

The producer injects traceparent into the message headers; the consumer extracts it and starts a span linked to the producer's span, so the trace continues across the async hop.

Q4. Checkout is slow and each team says it's not them. What do you do? SCENARIO

Open traces for slow checkouts, read the waterfall to find the longest sequential spans, and fix that path, e.g. batching repeated calls.

Q5. Traces stop at one service even though it's instrumented. Likely causes? TRICKY

Context lost in a thread pool or async callback, a client that doesn't forward headers, or a proxy stripping them.

Q6. How do you keep tracing affordable but still catch problems? TRICKY

Tail sampling: keep all error and slow traces and a small percentage of normal ones, decided after the trace completes.

Next: SD-L-019 · Microservice Anti-Patterns backendroadmap.com